

Daily Project1 Submission

In []:

Sonar Rock vs Mine Detection Using Machine Learning

In []:

1. Problem Statement

* The objective of this project is to build a machine learning model that can automatically classify underwater objects as either a **Rock (R)** or a **Naval Mine (M)**. The classification is based on sonar data, which measures the strength of sonar returns at different angles.*

In []:

2. Dataset Information

* Source: UCI Machine Learning Repository (Sonar, Mines vs. Rocks Dataset).

* Instances: 208 observations.

* Attributes: 60 numerical features representing sonar energy returns at different frequencies/angles.

* Target: 1 categorical column with two possible values:

- M: Mine (Object is a metal cylinder)
- R: Rock (Object is a rock)

* Dataset Type: Classification

```
In [1]: import os  
os.getcwd()
```

```
Out[1]: 'D:\\machine learning'
```

```
In [2]: import os  
os.listdir()
```

```
Out[2]: ['.ipynb_checkpoints',
        '1 What is Data.docx',
        '1stProjectML_Sonarnishaaa.pdf',
        '2 ML_Notes.docx',
        '2 ML_Notes.pdf',
        '2105COSandhyaMagadum.ipynb',
        '2105COSandhyaMagadum1.ipynb',
        '3.1.NumPy_Complete_Guide.pdf',
        '3.2 pandas complete guide.pdf',
        '3.3Seaborn_Complete_Training_Notebook.pdf',
        '4.Data_Cleaning_Missing_Values_Outliers.pdf',
        '5.Matplotlib_Tutorial.pdf',
        '6 Feature_Encoding_Complete.pdf',
        '8 feature_engineering using titanic dataset.pdf',
        '9 titanic_feature_train_testsplit.pdf',
        'Advertising.csv',
        'All Datasets',
        'All Datasets.zip',
        'B3 Parkinson Dataset.zip',
        'B6 ECG Arthamania',
        'B6 ECG Arthamania.zip',
        'B9 PCB Fault detection',
        'B9 PCB Fault detection.zip',
        'Bahaman.xlsx',
        'Bolt diameter.xlsx',
        'Boston.csv',
        'claimants sample.csv',
        'claimants.csv',
        'Contract Renewal.xlsx',
        'dailyproject1.ipynb',
        'Fabric data.xlsx',
        'gene_expression.csv',
        'hearing_test.csv',
        'homeprices.csv',
        'Indian Liver Patient Dataset -Description.txt',
        'JohnnyTalkers.xlsx',
        'liver.csv',
        'liver.xlsx',
        'liver_synthetic.csv',
        'liver_synthetic.json',
        'liver_synthetic.xlsx',
        'machine learning',
        'model',
        'mouse_viral_study.csv',
        'mushrooms.csv',
        'penguins_size.csv',
        'Promotion.xlsx',
        'Seaborn.ipynb',
        'sonar data.csv',
        'stroke prediction.csv',
        'student_data.csv',
        'student_data.json',
        'student_data.xlsx',
        'Task17_7.pdf',
        'task23.ipynb',
        'taskPlots.ipynb',
```

```
'Task_2105.docx',  
'tips.csv',  
'titanic.csv',  
'train_test.csv',  
'WorldCupMatches.csv']
```

```
In [3]: import os  
os.listdir("All Datasets")
```

```
Out[3]: ['Advertising.csv',  
'Bahaman.xlsx',  
'Bolt diameter.xlsx',  
'Boston.csv',  
'claimants sample.csv',  
'claimants.csv',  
'Contract Renewal.xlsx',  
'Fabric data.xlsx',  
'gene_expression.csv',  
'hearing_test.csv',  
'homeprices.csv',  
'Indian Liver Patient Dataset -Description.txt',  
'JohnnyTalkers.xlsx',  
'liver.csv',  
'liver.xlsx',  
'mouse_viral_study.csv',  
'mushrooms.csv',  
'penguins_size.csv',  
'Promotion.xlsx',  
'stroke prediction.csv',  
'tips.csv',  
'titanic.csv',  
'train_test.csv']
```

```
In [ ]:
```

3. Data Loading

```
In [4]: import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
In [5]: df = pd.read_csv("sonar data.csv" , header=None)
```

```
In [6]: df
```

Out[6]:

	0	1	2	3	4	5	6	7	8	9	...	
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.00
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.00
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.02
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.01
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.00
...	...	...	...	...	...	...	...	...	...	...	...	...
203	0.0187	0.0346	0.0168	0.0177	0.0393	0.1630	0.2028	0.1694	0.2328	0.2684	...	0.01
204	0.0323	0.0101	0.0298	0.0564	0.0760	0.0958	0.0990	0.1018	0.1030	0.2154	...	0.00
205	0.0522	0.0437	0.0180	0.0292	0.0351	0.1171	0.1257	0.1178	0.1258	0.2529	...	0.01
206	0.0303	0.0353	0.0490	0.0608	0.0167	0.1354	0.1465	0.1123	0.1945	0.2354	...	0.00
207	0.0260	0.0363	0.0136	0.0272	0.0214	0.0338	0.0655	0.1400	0.1843	0.2354	...	0.01

208 rows × 61 columns



In [7]:

```
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 208 entries, 0 to 207
```

```
Data columns (total 61 columns):
```

#	Column	Non-Null Count	Dtype
0	0	208 non-null	float64
1	1	208 non-null	float64
2	2	208 non-null	float64
3	3	208 non-null	float64
4	4	208 non-null	float64
5	5	208 non-null	float64
6	6	208 non-null	float64
7	7	208 non-null	float64
8	8	208 non-null	float64
9	9	208 non-null	float64
10	10	208 non-null	float64
11	11	208 non-null	float64
12	12	208 non-null	float64
13	13	208 non-null	float64
14	14	208 non-null	float64
15	15	208 non-null	float64
16	16	208 non-null	float64
17	17	208 non-null	float64
18	18	208 non-null	float64
19	19	208 non-null	float64
20	20	208 non-null	float64
21	21	208 non-null	float64
22	22	208 non-null	float64
23	23	208 non-null	float64
24	24	208 non-null	float64
25	25	208 non-null	float64
26	26	208 non-null	float64
27	27	208 non-null	float64
28	28	208 non-null	float64
29	29	208 non-null	float64
30	30	208 non-null	float64
31	31	208 non-null	float64
32	32	208 non-null	float64
33	33	208 non-null	float64
34	34	208 non-null	float64
35	35	208 non-null	float64
36	36	208 non-null	float64
37	37	208 non-null	float64
38	38	208 non-null	float64
39	39	208 non-null	float64
40	40	208 non-null	float64
41	41	208 non-null	float64
42	42	208 non-null	float64
43	43	208 non-null	float64
44	44	208 non-null	float64
45	45	208 non-null	float64
46	46	208 non-null	float64
47	47	208 non-null	float64
48	48	208 non-null	float64
49	49	208 non-null	float64
50	50	208 non-null	float64

```
51 51      208 non-null    float64
52 52      208 non-null    float64
53 53      208 non-null    float64
54 54      208 non-null    float64
55 55      208 non-null    float64
56 56      208 non-null    float64
57 57      208 non-null    float64
58 58      208 non-null    float64
59 59      208 non-null    float64
60 60      208 non-null    object
dtypes: float64(60), object(1)
memory usage: 99.3+ KB
```

```
In [8]: df.describe()
```

Out[8]:

	0	1	2	3	4	5	6
count	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000	208.000000
mean	0.029164	0.038437	0.043832	0.053892	0.075202	0.104570	0.121747
std	0.022991	0.032960	0.038428	0.046528	0.055552	0.059105	0.061788
min	0.001500	0.000600	0.001500	0.005800	0.006700	0.010200	0.003300
25%	0.013350	0.016450	0.018950	0.024375	0.038050	0.067025	0.080900
50%	0.022800	0.030800	0.034300	0.044050	0.062500	0.092150	0.106950
75%	0.035550	0.047950	0.057950	0.064500	0.100275	0.134125	0.154000
max	0.137100	0.233900	0.305900	0.426400	0.401000	0.382300	0.372900

8 rows × 60 columns



```
In [10]: df[60].value_counts()
```

```
Out[10]: 60
M      111
R       97
Name: count, dtype: int64
```

```
In [11]: df.columns
```

```
Out[11]: Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
                18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
                36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53,
                54, 55, 56, 57, 58, 59, 60],
                dtype='int64')
```

```
In [ ]:
```

```
In [ ]:
```

4. Data Cleaning

4.1] Check Missing Values

```
In [12]: df.isnull().sum()
```

```
Out[12]: 0      0
         1      0
         2      0
         3      0
         4      0
         ..
        56      0
        57      0
        58      0
        59      0
        60      0
        Length: 61, dtype: int64
```

```
In [ ]:
```

4.2] Remove Duplicate Records

```
In [13]: df.duplicated().sum()
```

```
Out[13]: 0
```

```
In [ ]:
```

4.3] Correct Data Types

```
In [14]: df.dtypes
```

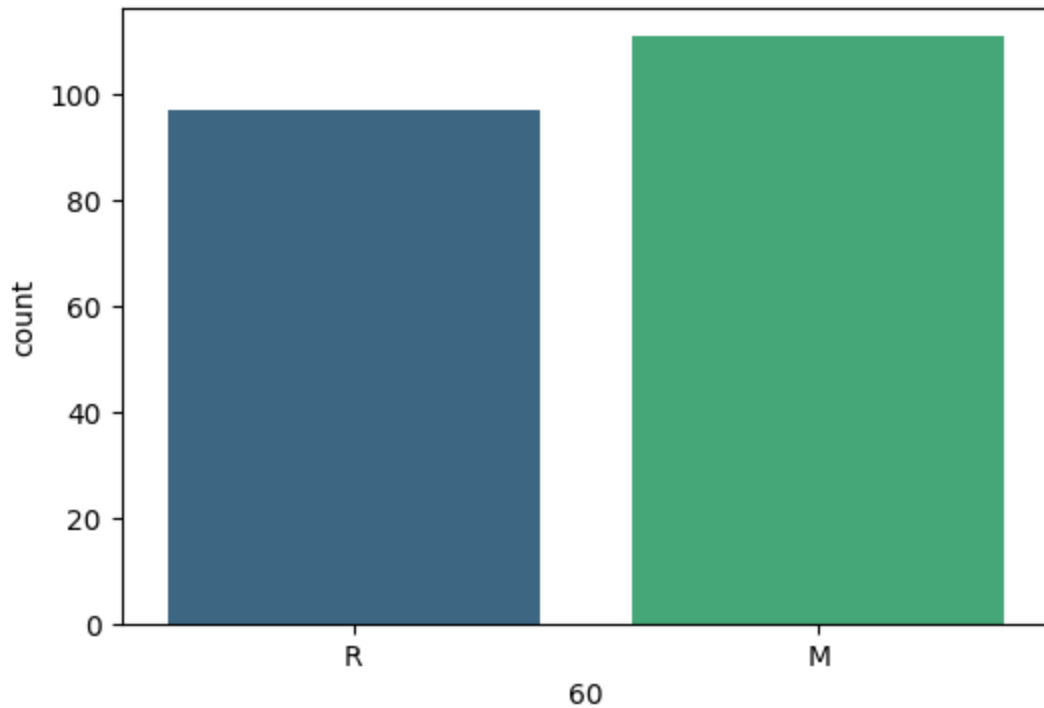
```
Out[14]: 0      float64
         1      float64
         2      float64
         3      float64
         4      float64
         ...
        56      float64
        57      float64
        58      float64
        59      float64
        60      object
        Length: 61, dtype: object
```

```
In [ ]:
```

5. Exploratory Data Analysis (EDA)

* Count Plot

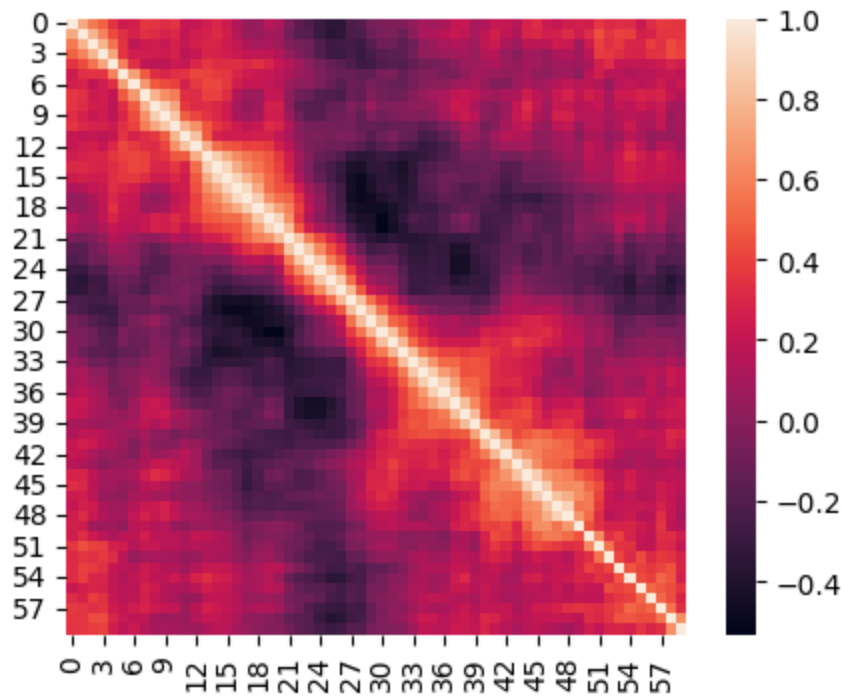
```
In [15]: plt.figure(figsize=(6, 4))  
  
sns.countplot(x=df.iloc[:, -1], hue=df.iloc[:, -1], palette='viridis', legend=False)  
  
plt.show()
```



In []:

* Correction Heatmap

```
In [16]: plt.figure(figsize=(5, 4))  
  
sns.heatmap(df.corr(numeric_only=True))  
  
plt.show()
```



In []:

* Bar Plot

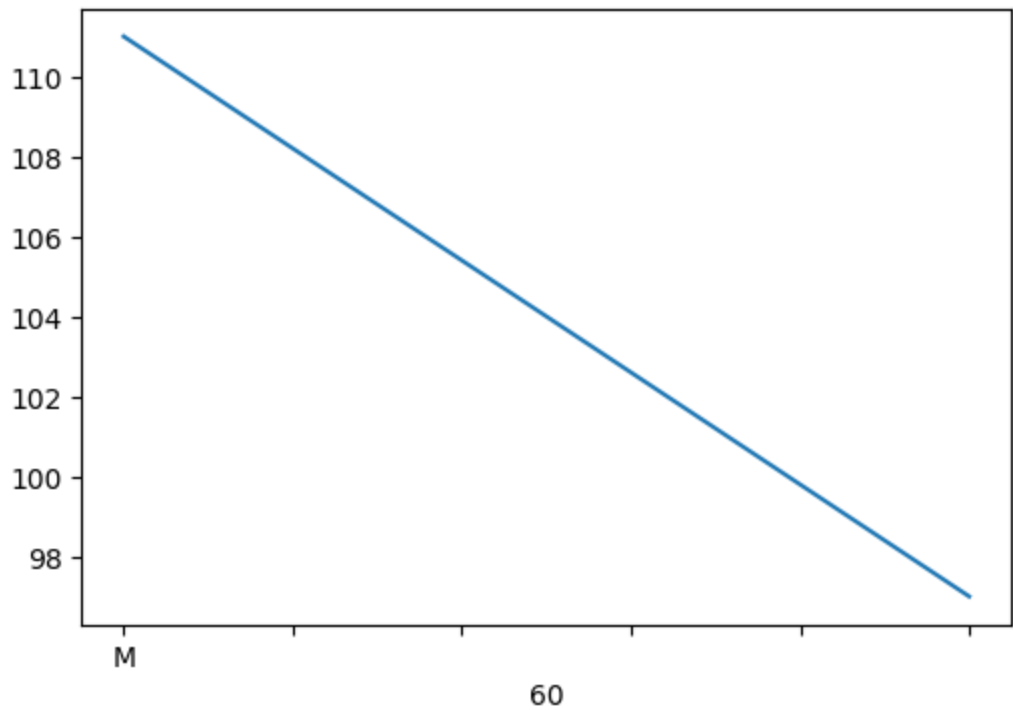
In [17]: `print(df.columns)`

```
Index([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
        36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53,
        54, 55, 56, 57, 58, 59, 60],
      dtype='int64')
```

```
In [18]: plt.figure(figsize=(6, 4))

df.iloc[:, 60].value_counts().plot()

plt.show()
```



```
In [19]: df.tail()
```

Out[19]:

	0	1	2	3	4	5	6	7	8	9	...	
203	0.0187	0.0346	0.0168	0.0177	0.0393	0.1630	0.2028	0.1694	0.2328	0.2684	...	0.01
204	0.0323	0.0101	0.0298	0.0564	0.0760	0.0958	0.0990	0.1018	0.1030	0.2154	...	0.00
205	0.0522	0.0437	0.0180	0.0292	0.0351	0.1171	0.1257	0.1178	0.1258	0.2529	...	0.01
206	0.0303	0.0353	0.0490	0.0608	0.0167	0.1354	0.1465	0.1123	0.1945	0.2354	...	0.00
207	0.0260	0.0363	0.0136	0.0272	0.0214	0.0338	0.0655	0.1400	0.1843	0.2354	...	0.01

5 rows × 61 columns

```
In [ ]:
```

```
In [ ]:
```

7. Feature Engineering (if applicable)

X and y Separation:

```
In [21]: X=df.drop(columns=[60],axis=1)
y=df[60]
```

```
In [ ]:
```

Feature Scalling

```
In [22]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [23]: print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

(166, 60)

(42, 60)

(166,)

(42,)

```
In [24]: df.head()
```

```
Out[24]:
```

	0	1	2	3	4	5	6	7	8	9	...	51
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031

5 rows × 61 columns



```
In [26]: X = df.drop(columns=60)
y = df[60]
```

```
In [27]: X.head()
```

```
Out[27]:
```

	0	1	2	3	4	5	6	7	8	9	...	50
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0232
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0125
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0033
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0241
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0156

5 rows × 60 columns



```
In [28]: y.head()
```

```
Out[28]: 0    R
          1    R
          2    R
          3    R
          4    R
          Name: 60, dtype: object
```

```
In [29]: print(X.shape)
          print(y.shape)
```

```
(208, 60)
(208,)
```

```
In [ ]:
```

8 Model Building

Train-Test Split

The dataset is divided into training and testing data.

- Training Data : 80%

- Testing Data : 20

```
In [30]: from sklearn.model_selection import train_test_split
          X_train, X_test, y_train, y_test = train_test_split(
              X, y, test_size=0.2, random_state=42
          )
```

```
In [31]: print(X_train.shape)
          print(X_test.shape)
          print(y_train.shape)
          print(y_test.shape)
```

```
(166, 60)
(42, 60)
(166,)
(42,)
```

```
In [33]: from sklearn.linear_model import LogisticRegression
```

Model craeting

```
In [34]: model = LogisticRegression()
```

Model Training

```
In [35]: model.fit(X_train, y_train)
```

```
Out[35]:
```

▼ LogisticRegression ⓘ ?

- Parameters
- Fitted attributes

Logistic Regression model is used for classification, The model is trained using the training dataset.

```
In [ ]:
```

```
In [ ]:
```

9. Model Evaluation

Prediction

```
In [36]: train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
```

```
In [37]: train_predictions
```

```
Out[37]: array(['R', 'M', 'R', 'M', 'M', 'M', 'R', 'R', 'R', 'M', 'R', 'M', 'M',
                'M', 'M', 'M', 'R', 'M', 'M', 'M', 'R', 'M', 'R', 'R',
                'R', 'R', 'R', 'R', 'R', 'M', 'R', 'M', 'M', 'R', 'R', 'M', 'R',
                'M', 'R', 'M', 'M', 'M', 'M', 'R', 'R', 'M', 'M', 'R', 'R', 'M',
                'R', 'M', 'R', 'R', 'R', 'M', 'M', 'R', 'M', 'M', 'R', 'R', 'M',
                'M', 'M', 'R', 'R', 'R', 'M', 'R', 'M', 'M', 'R', 'R', 'M', 'M',
                'M', 'R', 'M', 'R', 'R', 'M', 'R', 'M', 'M', 'R', 'M', 'R', 'M',
                'M', 'M', 'M', 'M', 'M', 'R', 'M', 'M', 'R', 'M', 'M', 'M', 'R',
                'M', 'R', 'R', 'M', 'M', 'R', 'R', 'M', 'M', 'M', 'M', 'R', 'R',
                'R', 'R', 'M', 'M', 'M', 'R', 'R', 'R', 'R', 'R', 'M', 'M', 'M',
                'R', 'M', 'M', 'R', 'R', 'M', 'M', 'R', 'R', 'R', 'M', 'R', 'R',
                'M', 'M', 'M', 'M', 'R', 'R', 'R', 'R', 'M', 'R'], dtype=object)
```

```
In [ ]:
```

Evaluations

```
In [38]: from sklearn.metrics import accuracy_score

print("Train Accuracy :", accuracy_score(y_train, train_predictions))
print("Test Accuracy :", accuracy_score(y_test, test_predictions))
```

Train Accuracy : 0.8373493975903614

Test Accuracy : 0.7857142857142857

```
In [39]: from sklearn.metrics import confusion_matrix
         confusion_matrix(y_test, test_predictions)
```

```
Out[39]: array([[19,  7],
               [ 2, 14]], dtype=int64)
```

```
In [41]: from sklearn.metrics import classification_report
         print(classification_report(y_test, test_predictions))
```

	precision	recall	f1-score	support
M	0.90	0.73	0.81	26
R	0.67	0.88	0.76	16
accuracy			0.79	42
macro avg	0.79	0.80	0.78	42
weighted avg	0.81	0.79	0.79	42

```
In [ ]:
```

10. Conclusion

The machine learning model successfully detects underwater rocks and mines.

Accuracy Gap: Evaluated at 83.73% training vs 78.57% testing.

Sample Volume: Validated using 42 distinct sonar signals.

Mine Classification: Earned 90% precision but only 73% recall.

Rock Classification: Logged 67% precision and 88% recall.

Critical Errors: Discovered 7 dangerous false-negative mine misses.

In []:

In []:

In []: